ARCHITECTURE OVERVIEW

Virtualization vs Containers

The Evolution of Application Deployment

Before Virtualization

- 1 Application per physical server
- Low hardware utilization
- Expensive infrastructure
- Difficult scaling
- Resource waste

Virtualization Introduced

- Multiple Virtual Machines (VMs) on 1 server
- Better resource utilization
- Isolation between workloads
- Easier infrastructure management

Containers Evolved

- Faster startup times
- Smaller resource footprint
- Better portability across environments
- Designed for cloud-native deployments

Speaker Notes: *Virtualization solved the problem of inefficient hardware usage. Containers took the next step by virtualizing the operating system instead of the hardware, making applications much lighter and faster to deploy.*

Virtual Machines vs Containers

FEATURE / METRIC	VIRTUAL MACHINES (VMS)	CONTAINERS
Abstraction Level	Virtualize physical hardware	Virtualize operating system
Operating System	Each VM has its own Guest OS	Share host OS kernel
Footprint Size	Larger (Gigabytes / GBs)	Smaller (Megabytes / MBs)
Startup Speed	Startup in minutes	Startup in seconds
Resource Overhead	Higher resource usage	Lightweight efficiency
Isolation Level	Strong hardware isolation	Process-level isolation

VM STACK

VM 1 (OS + App) | VM 2 (OS + App)

Hypervisor

Physical Server

CONTAINER STACK

Container 1 | Container 2 | Container 3

Container Runtime (e.g. Docker)

Host OS / Hardware

Key Takeaway: VMs package an entire operating system. Containers package only the application and its explicit dependencies.

Speaker Notes: *Walk through the architectural comparison: Physical Server → Hypervisor → VMs (OS+App) vs Host OS → Container Runtime → Containers.*

From Monoliths to Cloud-Native

Traditional Monolithic Apps

- ✕ **Single Codebase:** Everything compiled together.
- ✕ **Single Deployment:** Changes require full re-deploy.
- ✕ **Rigid Scaling:** Scaling means scaling entire stack.
- ✕ **Cascading Failures:** One failure can impact whole app.

Modern Cloud-Native Architecture

- ✓ **Microservices:** Decoupled modular services.
- ✓ **Independent Deployments:** Faster release cycles.
- ✓ **API Communication:** REST / gRPC messaging.
- ✓ **Horizontal Scaling & Fault Isolation:** High availability.

Cloud-Native Core Characteristics: CONTAINERIZED | AUTOMATED | OBSERVABLE | RESILIENT | SCALABLE

Speaker Notes: *Mention Netflix, Uber, and Spotify as prime examples where independent microservices (auth, payments, recommendations, notifications) run in isolated containers.*

Introduction to Docker

What is Docker? Docker is a platform that packages applications and all their dependencies into standardized containers.

Docker Benefits

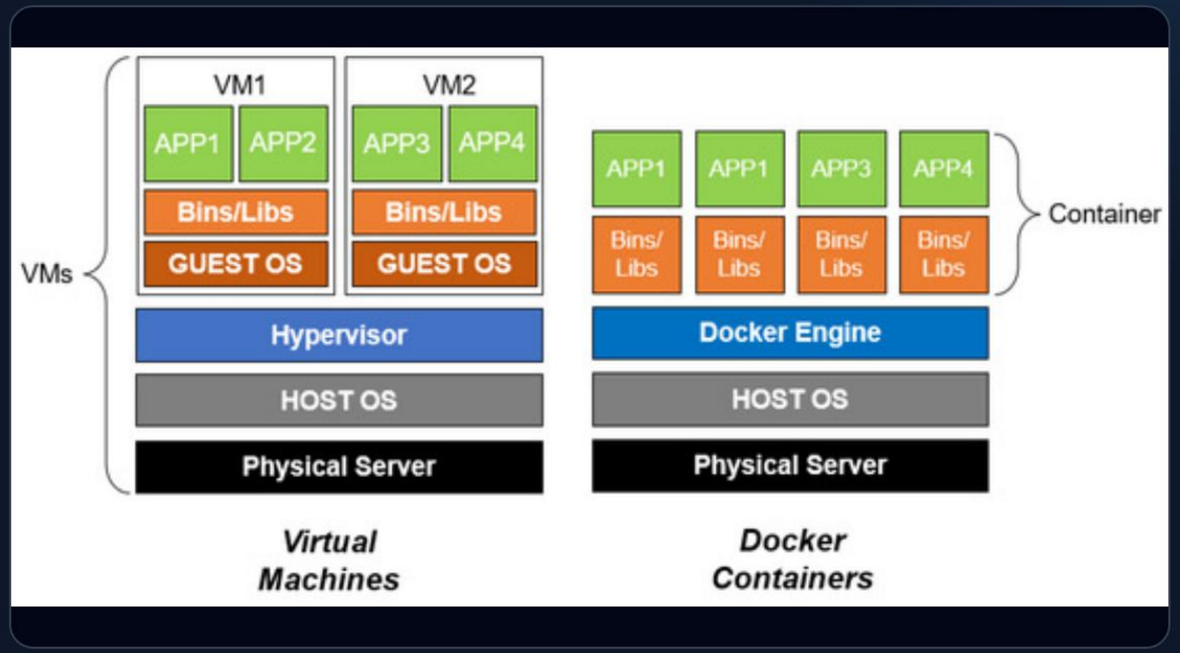
- ✓ Consistent dev, test, and prod environments
- ✓ Highly portable across cloud & on-prem machines
- ✓ Fast deployments and lightweight footprint
- ✓ Easy dependency management

Core Components

`Docker Engine` • `Images` • `Containers` • `Dockerfile` • `Docker Hub`

STANDARD WORKFLOW

`Write Code → Build Image → Run Container → Deploy Anywhere`



Speaker Notes: *"Docker doesn't replace your application—it packages it so it behaves the same on your laptop, in testing, and in production environments."*

Building Your First Containerized App

1. Typical Workflow

1. Create application
2. Write a `Dockerfile`
3. Build Docker image
4. Run container
5. Test app & share image

2. Example Projects

- **Python Flask API:** Serves `Hello, World!`
- **Node.js Express:** Serves `GET / → Hello Workshop!`

3. Why Containerize?

Eliminates dependency conflicts, provides seamless team sharing, and guarantees identical environment runtime everywhere.

Sample Dockerfile Structure

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt . && RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Speaker Notes: *Emphasize that the application code itself isn't changing—only the packaging, delivery, and execution model are changing.*

Running Containers on Your Machine

`docker pull nginx`
Download an image template from Docker Hub registry.

`docker images`
List all container images saved locally on host.

`docker run nginx`
Create and start a new running container instance.

`docker ps`
View all currently running container processes.

`docker stop`
Stop execution of a running container gracefully.

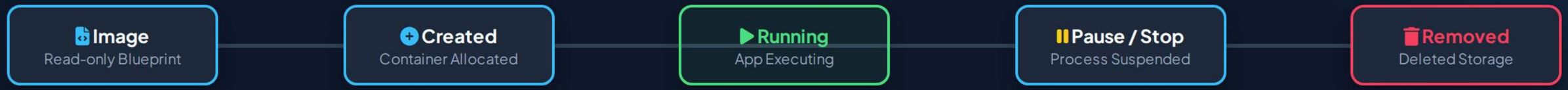
`docker rm`
Remove a stopped container from local disk storage.

Essential Mental Model

Images are read-only blueprints / templates. **Containers** are active running instances created from those templates.

Speaker Notes: *Demonstrate each command sequentially in terminal: `pull` → `images` → `run` → `ps` → `stop` → `rm` to show full container management cycle.*

Understanding Container Lifecycle



⚠ Critical Concept: Ephemerality

Containers are short-lived by design. Orchestrators like Kubernetes or OpenShift destroy and recreate failed containers automatically. Persistent data must live outside containers (volumes, DBs, cloud storage).

SUGGESTED DEMO FLOW (30–40 MINS)

1. Virt vs Containers (5m)
2. Modern Arch (5m)
3. Docker Intro (5m)
4. Build & Run App (20m)

Speaker Notes: *This is a crucial concept before introducing Kubernetes or OpenShift. In production, if a container crashes, orchestrators recreate a new instance rather than repairing the old container.*

Image Sources

https://www.mdpi.com/applsci/applsci-12-06737/article_deploy/html/images/applsci-12-06737-g001-550.jpg

Source: www.mdpi.com